

Composição, Agregação e Associação

Como as classes se relacionam

Até agora, criamos classes que guardam **referências** para outras classes dentro de seus atributos.

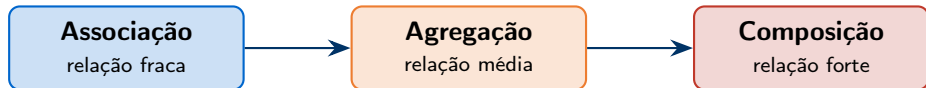
Mas nem toda referência é igual

Um Motorista que dirige um Carro não tem a mesma relação que um Carro que possui um Motor.

Hoje vamos aprender a diferenciar **três tipos** de relação "tem um": **associação**, **agregação** e **composição**.

Três níveis de força

As três relações formam um espectro, do mais fraco ao mais forte:



Vamos ver cada uma com exemplos bem simples.

O que é uma associação?

Definição

Uma **associação** acontece quando uma classe apenas **conhece** ou **usa** outra, sem que nenhuma das duas seja "dona" da outra.

- as duas classes **existem de forma independente**;
- a referência pode mudar a qualquer momento;
- é o tipo de relação **mais fraco** entre objetos.

Exemplo: Motorista e Carro

Um Motorista **usa** um Carro, mas nenhum dos dois pertence ao outro:

```
public class Carro {
    private String modelo;

    public Carro(String modelo) {
        this.modelo = modelo;
    }
}

public class Motorista {
    private String nome;
    private Carro carroAtual;

    public Motorista(String nome) {
        this.nome = nome;
    }

    public void dirigir(Carro carro) {
        this.carroAtual = carro;
    }
}
```

Características da associação

- o Carro é criado **fora** do Motorista, e só chega até ele pelo método dirigir;
- o mesmo Motorista pode dirigir **vários carros diferentes** ao longo do tempo;
- o mesmo Carro pode ser dirigido por **vários motoristas diferentes**;
- se o objeto Motorista deixar de existir, o Carro continua existindo normalmente – e vice-versa.

Resumindo

Associação é só uma **referência** entre duas classes que vivem de forma independente.

O que é uma agregação?

Definição

Uma **agregação** é uma associação mais forte: uma classe representa um **todo** que **reúne** partes, mas essas partes têm vida própria.

- o "todo" **reúne** as partes, mas não é quem as criou;
- cada parte pode existir **antes** de entrar no todo, e continuar existindo **depois** de sair dele.

Exemplo: Biblioteca e Livro

Uma Biblioteca **reúne** vários Livros:

```
public class Livro {  
    private String titulo;  
  
    public Livro(String titulo) {  
        this.titulo = titulo;  
    }  
}  
  
public class Biblioteca {  
    private List<Livro> livros = new ArrayList<>();  
  
    public void adicionar(Livro livro) {  
        livros.add(livro);  
    }  
}
```

Repare: o Livro é criado **em algum lugar de fora** e só depois **entregue** para a Biblioteca guardar.

Biblioteca é uma composição de livros?

Não! É uma agregação.

Um Livro **não nasce dentro** da Biblioteca – ele é escrito, comprado ou ganhado, e só **depois** entra na biblioteca.

- o livro pode existir **antes** de entrar na biblioteca;
- o livro pode ser **removido** e continuar existindo em outro lugar (emprestado, doado, vendido);
- se a biblioteca fechar, os **livros não deixam de existir**.

Regra prática da agregação

Pergunta para fazer

"Se eu tirar essa parte do todo, ela continua fazendo sentido sozinha?"

Se a resposta for "sim"...

...então a relação é uma **agregação**: a parte **sobrevive** sem o todo, mesmo fazendo parte dele agora.

O que é uma composição?

Definição

Uma **composição** é a relação "tem um" mais **forte** que existe: o todo **cria** e **controla** completamente o ciclo de vida das partes.

- a parte **nasce junto** com o todo;
- a parte **não faz sentido sozinha**, fora do todo;
- se o todo for destruído, a parte **vai junto**.

Exemplo: Carro e Motor

Um Carro **cria** o seu próprio Motor:

```
public class Motor {  
    private int potencia;  
  
    public Motor(int potencia) {  
        this.potencia = potencia;  
    }  
}  
  
public class Carro {  
    private final Motor motor;  
  
    public Carro(String modelo, int potenciaMotor) {  
        this.motor = new Motor(potenciaMotor);  
    }  
}
```

Repare: ninguém de fora entrega um Motor pronto para o Carro – é o próprio Carro quem cria o seu, lá dentro do construtor.

Por que isso é composição, e não agregação?

- o Motor **nasce dentro** do construtor de Carro, com `new Motor(...)`;
- não existe nenhum `setMotor(...)` recebendo um motor pronto de fora;
- não existe nenhum `getMotor()` devolvendo o motor para ser reaproveitado em outro carro.

Regra prática

Se o **todo cria** a parte, e ninguém de fora consegue pegar essa parte pronta, é **composição**.

Carro e suas partes

A mesma ideia vale para **várias** partes de uma vez – como as rodas do carro:

```
public class Roda {
    private int aro;

    public Roda(int aro) {
        this.aro = aro;
    }
}

public class Carro {
    private final Motor motor;
    private final List<Roda> rodas = new ArrayList<>();

    public Carro(String modelo, int potenciaMotor) {
        this.motor = new Motor(potenciaMotor);
        for (int i = 0; i < 4; i++) {
            rodas.add(new Roda(17));
        }
    }
}
```

Associação, agregação e composição lado a lado

	Associação	Agregação	Composição
Quem cria a parte?	de fora	de fora	o próprio todo
Parte sobrevive sem o todo?	sim	sim	não
Força da relação	fraca	média	forte
Exemplo	Motorista–Carro	Biblioteca–Livro	Carro–Motor

Como identificar rapidamente

Pergunta 1

A classe A guarda uma referência para B? Se não houver nenhuma referência, não existe relação "tem um".

Pergunta 2

B é criado **fora** de A e só chega pronto (por parâmetro ou set)? Então é **associação** ou **agregação** – depende se B é só um colaborador (associação) ou uma parte que compõe A (agregação).

Pergunta 3

B é criado **dentro** do construtor de A, sem que ninguém de fora consiga pegá-lo pronto? Então é **composição**.

- **Associação**: duas classes só se **conhecem**, e vivem de forma totalmente independente (Motorista e Carro);
- **Agregação**: uma classe **reúne** partes criadas de fora, que **sobrevivem** sem o todo (Biblioteca e Livro);
- **Composição**: uma classe **cria** e **controla** suas partes, que **não existem** sem o todo (Carro e Motor);
- a pergunta-chave é sempre a mesma: **quem cria a parte, e ela sobrevive sem o todo?**